Conocimiento y Razonamiento Automatizado

PECL1

Representación y Simplificación de Sudokus en Prolog

Grado en Ingeniería Informática
Universidad de Alcalá

Álvaro Fernández Narciso
Andrea Pascual Aguilera
Sergio Alonso Zarcero

Marzo 2024-2025

Resumen

Este trabajo presenta el desarrollo de un sistema en Prolog para la representación y simplificación de Sudokus, basado en el manejo de listas y la aplicación de reglas de reducción de dominio. El Sudoku se modela como una lista de 81 elementos, donde se generan listas de posibilidades para casillas vacías y se implementan cuatro reglas de simplificación (Regla 0, Regla 1, Regla 2 y Regla 3) que reducen progresivamente las opciones mediante restricciones del juego. Estas reglas se aplican iterativamente hasta estabilizar el tablero, con validación en un conjunto de 24 Sudokus de niveles básico, intermedio y avanzado, además de casos especiales (vacío e irresoluble).

Los resultados, analizados mediante métricas detalladas, muestran que el sistema resuelve completamente Sudokus básicos e intermedios (100% de éxito), pero presenta limitaciones en los avanzados (23.18% de casillas resueltas, 25% resueltos), destacando la eficacia de la Regla 0 (29.21%) y la menor contribución de las Reglas 2 y 3. Se comparan dos flujos de aplicación (normal e inverso), siendo el inverso más eficiente en iteraciones (41 vs. 50), aunque con igual tasa de resolución (66.67%). Mejoras como una interfaz ASCII, detección de conflictos y ejecución iterativa enriquecen la implementación.

En conclusión, el sistema demuestra competencia en casos simples y medianos, pero requiere técnicas avanzadas, como backtracking, para abordar Sudokus complejos. Este proyecto refuerza habilidades en Prolog, como recursividad y gestión de listas, y sienta bases para futuras extensiones.

Comandos

Sudokus disponibles: sudoku_BASICO_X, sudoku_INTERMEDIO_X, sudoku_AVANZADO_X, sudoku_MALO y sudoku_VACIO.

X es un número del 1 al 8.

Comandos desde main.pl:

resolver(*nombre_sudoku*)

mostrarPosibilidades(*nombre_sudoku*)

Comandos desde métricas.pl:

probar_metricas(*nombre_sudoku*).

probar_metricas_globales.

probar_metricas_globales_basicos.

probar_metricas_globales_intermedios

probar_metricas_globales_avanzados.

Índice

Resumen *1*

Introducción *4*

Objetivos *4*

Desarrollo *5*

Representación del Sudoku **5**

Lista de Posibilidades **6**

Generación de Posibilidades 6

Impresión de Posibilidades 6

Implementación de las Reglas de Simplificación **7**

Regla 0 7

Regla 1 7

Regla 2 8

Regla 3 8

Ejecución y Comprobación de Simplificación **8**

Validación del Conjunto de Ejemplos **9**

Otros aspectos relevantes de la implementación **10**

Mejoras y Extensiones *11*

1. Interfaz ASCII **11**

2. Detección de conflictos **12**

3. Iteración **13**

Métricas *13*

Fuentes consultadas *19*

Distribución de tareas *20*

Conclusiones *21*

Anexos *22*

Anexo 1: Lista de Variables **22**

Anexo 2: Lista de funciones **24**

Introducción

El Sudoku es un juego lógico basado en una cuadrícula de 9x9 en la que los números deben colocarse siguiendo ciertas restricciones. Aunque existen enfoques matemáticos avanzados para resolver estos rompecabezas, en esta práctica se explora un método basado en listas en Prolog para la representación y simplificación de Sudokus.

El propósito de este trabajo es desarrollar un sistema que, dado un Sudoku inicial, genere una lista de posibles valores para cada casilla vacía y aplique una serie de reglas de simplificación para reducir progresivamente las posibilidades. Estas reglas no garantizan la resolución completa del Sudoku, pero pueden simplificarlo significativamente.

La práctica permite poner en práctica conceptos fundamentales de manejo de listas en Prolog, uso de predicados recursivos y aplicación de estrategias de reducción de dominio. Además, se explorará la efectividad de estas reglas en distintos niveles de dificultad y se analizará la posibilidad de extenderlas con mejoras adicionales.

Este informe documenta la metodología utilizada, las reglas implementadas, los resultados obtenidos y las posibles mejoras que podrían aplicarse para hacer el sistema más eficiente.

Objetivos

El objetivo principal de esta práctica es desarrollar una implementación en Prolog que permita representar un Sudoku, generar posibles valores para cada casilla vacía y aplicar reglas de simplificación para reducir el espacio de soluciones.

Para lograr esto, se plantean los siguientes objetivos específicos:

1. **Representación del Sudoku**: Implementar una estructura de datos en Prolog basada en listas de longitud 81 para modelar el tablero de Sudoku.

2. **Generación de posibilidades**: Crear un algoritmo que, dado un Sudoku inicial, genere la lista de valores posibles para cada casilla vacía.

3. **Aplicación de reglas de simplificación**: Implementar y probar cuatro reglas que permitan reducir las listas de posibilidades de cada casilla mediante restricciones del juego.

4. **Validación con ejemplos**: Evaluar el sistema utilizando al menos cinco Sudokus de diferentes niveles de dificultad, midiendo la efectividad de las reglas de simplificación.

5. **Propuesta de mejoras**: Analizar los casos donde las reglas implementadas no logran resolver el Sudoku completamente y plantear extensiones o técnicas adicionales que puedan mejorar el rendimiento.

6. **Optimización del código**: Organizar el código de forma modular para facilitar su mantenimiento y extensión.

Desarrollo

Representación del Sudoku

El sudoku se representa como una lista de 81 elementos, donde cada posición de la lista corresponde a una casilla del tablero de 9×9. Las casillas vacías se representan mediante el número 0, mientras que los valores ya asignados en la cuadrícula se mantienen como números del 1 al 9.

Para la impresión del tablero se usa el predicado *imprimirTablero*, que recibe una lista de 81 elementos (un sudoku) y la imprime en filas de 9 elementos para mantener la estructura original del juego. Esto se logra recorriendo la lista e imprimiendo cada número. En caso de que la casilla contenga un 0 (indicando que está vacía), se imprime un "." en su lugar para facilitar la visualización de las casillas aún pendientes de resolver.

La impresión del tablero se gestiona a través del predicado auxiliar *imprimirFilas*, el cual funciona de manera recursiva siguiendo la siguiente lógica:

1. **Caso base:** Cuando el índice de la casilla actual alcanza el valor 81, el proceso termina, ya que se ha recorrido todo el tablero.

2. **Obtención del valor de la casilla:** Se usa nth0 para obtener el elemento correspondiente de la lista en la posición actual.

3. **Manejo de listas anidadas:** En algunos casos, el valor en la casilla puede ser una lista (por ejemplo, una lista de valores posibles en la fase de análisis de posibilidades). Si la casilla contiene una lista con un solo elemento, se extrae e imprime directamente el valor único. En caso contrario, se imprime la lista completa.

4. **Formato de impresión:**

- Si la casilla contiene un número distinto de 0, se imprime directamente.

- Si la casilla está vacía (0), se imprime un "." en su lugar para representar visualmente que la casilla aún no tiene un valor asignado.

5. **Salto de línea:** Para mantener el formato de un tablero 9x9, se inserta un salto de línea después de cada 9 elementos (I mod 9 =:= 8).

6. **Llamada recursiva:** Se incrementa el índice (NI is I + 1) y se llama recursivamente a *imprimirFilas* hasta completar la impresión del tablero.

Lista de Posibilidades

Generación de Posibilidades

Para resolver un sudoku, primero hay que determinar qué valores pueden ir en cada casilla vacía. Esto se hace con el predicado *hacerPosibilidades,* que genera una lista donde cada posición corresponde a una casilla del tablero e indica los números que pueden ir ahí respetando las reglas propias de un sudoku.

Los pasos a seguir son:

1. Se recorre todo el tablero, analizando casilla por casilla.

2. Si la casilla ya tiene un número asignado (1-9), simplemente se guarda ese número como una lista de un solo elemento [X].

3. Si la casilla está vacía (0), se calculan los valores posibles eliminando los que ya están en la misma fila, columna o cuadro 3x3.

4. La lista de posibilidades se va construyendo de manera recursiva hasta completar las 81 casillas.

El predicado *hacerPosibilidades* se encarga de:

- Cuando se llega a la casilla 81 (es decir, al final del tablero), terminar el proceso.

- Si la casilla ya tiene un número, mantenerlo tal cual en la lista.

- Si la casilla está vacía, calcular sus valores posibles restando los números que ya aparecen en su fila, columna y cuadro.

Impresión de Posibilidades

Una vez generadas las posibilidades, es importante mostrarlas de una forma clara. Para esto, se usa el predicado *imprimirPosibilidades*, que recorre toda la lista y muestra los valores posibles de cada casilla de esta forma:

1. Se recorre la lista de posibilidades usando *imprimirPosibilidadesAux*.

2. Se imprime el índice de la casilla y su contenido.

3. Si la casilla tiene solo un valor posible, se imprime solo ese número.

4. Si la casilla tiene varias opciones, se muestra la lista completa.

Aun así, al llevar a cabo la resolución de un sudoku se utiliza *imprimirTablero* después de *hacerPosibilidades* para mostrar un formato "más comprimido."

Implementación de las Reglas de Simplificación

Para reducir la cantidad de posibilidades en cada casilla y simplificar progresivamente el Sudoku, se implementan cuatro reglas de simplificación. Estas reglas se encuentran definidas en el archivo reglas.pl y main.pl hace uso de ellas gracias a ":-consult('reglas.pl')".

Regla 0

La regla 0 establece que, si una casilla tiene una única posibilidad, ese número se fija como su valor definitivo y se elimina de las casillas conflictivas (misma fila, columna y cuadro 3x3) mediante *quitarElementoDeConflictivos.*

El predicado regla0 recorre el tablero, verificando si la casilla ya fue procesada. Si no, obtiene su lista de posibilidades y, si tiene longitud 1, extrae su único valor. Este valor se fija como definitivo y elimina esa posibilidad de otras casillas con *quitarElementoDeConflictivos*. Luego, reinicia el proceso desde la primera casilla para seguir simplificando para así ir reduciendo las posibilidades en cada casilla.

Regla 1

La regla 1 detecta cuando un número es la única posibilidad en una casilla dentro de una fila, columna o cuadro 3x3 para fijarlo en esa casilla de manera definitiva. Además, se elimina de otras opciones. El predicado *regla1* recorre el tablero de posibilidades y para cada casilla, obtiene la lista de valores posibles con *nth0* y aplica *subregla1* para simplificarla según las diferentes condiciones: fila, columna y cuadro 3x3.

Si ninguna condición se cumple, se avanza al siguiente valor en la lista. El proceso se repite hasta que todo el tablero se haya recorrido y se devuelva el tablero actualizado, reduciendo las posibilidades de cada casilla.

Regla 2

La regla 2 se enfoca en detectar parejas de posibilidades que aparecen exactamente dos veces en una fila, columna o cuadro. Esto permite eliminar esas opciones en otras casillas dentro de la misma fila, columna o cuadro 3x3. Se realiza mediante el predicado *regla2*, que recorre el tablero de posibilidades y aplica la subregla subregla2 para simplificar las casillas que cumplen con estas condiciones. Este predicado evalúa las filas, columnas y cuadros y elimina esas opciones de las demás casillas de la fila mediante *quitarLista.*

En caso de no detectar ninguna pareja de este tipo, la subregla devuelve el tablero tal como está. Este proceso se repite de manera recursiva en el predicado *regla2* hasta que se hayan recorrido todas las casillas del tablero.

Regla 3

La regla 3 establece que, si en una fila, columna o cuadro contienen solo 3 números distintos como posibles candidatos, pudiéndose repetir las casillas. Si se dan estas condiciones, ninguno de los tres números aparecerá como candidato en ninguna casilla de esa fila, columna o cuadro.

Para aplicarla, la regla3 recorre el tablero de posibilidades buscando las casillas con 2 o 3 valores, candidatas a tríos. Cuando se encuentran, se verifica si puede formar un trío dentro del grupo y lo elimina.

La subregla3 se encarga de los dos escenarios posibles. En primer lugar, si hay dos posibilidades busca al tercer número para formar un trío y aplica la regla directamente. En caso de que se cumple, mediante puitarListaregla3 se eliminan los números del resto de casillas del grupo.

Para su aplicación, se utilizan funciones auxiliares como *comprobarCombinaciones* y *contarApariciones*, que aplica la subregla3 a los conjuntos de elementos resultantes tras realizar todas las posibles combinaciones y que cuentan cuántas veces aparece un conjunto de posibilidades, respectivamente.

Ejecución y Comprobación de Simplificación

La ejecución y comprobación del proceso de simplificación del sudoku se realiza mediante el predicado principal ***resolver*** que está definido en “main.pl”. Cuando se llama a este predicado con un sudoku inicial como argumento, se ejecuta la siguiente secuencia:

1. Se generan las posibilidades iniciales para cada casilla vacía utilizando el predicado *hacerPosibilidades*.

2. Después se verifica si el sudoku generado inicialmente, el que se pasa como argumento, es resoluble mediante el predicado *es_imposible*. Este predicado lo que hace es comprobar si el sudoku original es irresoluble porque tiene alguna casilla vacía, es decir, sin posibilidades, o porque tiene algún número repetido en la misma fila, columna o cuadrante 3x3. Si es irresoluble, se finaliza con un mensaje aclarando que el sudoku es irresoluble y, por tanto, imposible de resolver.

3. Se muestra por pantalla el sudoku inicial (el pasado como argumento), el sudoku inicial con posibilidades en las casillas vacías y también este último, pero en ASCII. Para estas impresiones de los sudokus se utiliza el predicado *imprimirTablero*.

4. Se aplican de forma secuencial las reglas de simplificación (0, 1, 2, y 3) mediante el predicado *simplificar_hasta_estabilizar*. Esta ejecución ocurre repetidamente hasta que se alcanza una estabilización, es decir, hasta que no se producen cambios en las listas de posibilidades de las casillas del tablero entre iteraciones consecutivas. A medida que se van aplicando las reglas, se van imprimiendo por pantalla las distintas iteraciones (pasos), junto con las reglas que se han aplicado en esa determinada iteración y el tablero del sudoku en ASCII.

5. Después de cada aplicación de reglas, se realiza una comprobación intermedia de validez del sudoku usando nuevamente el predicado *es_imposible*. En caso de detectarse una situación imposible, se detiene el proceso inmediatamente y se muestra por pantalla que es imposible de resolver.

6. Finalmente, si el sudoku está completamente resuelto, muestra el resultado final por pantalla como un tablero en formato ASCII.

Al fin y al cabo, está claro que *resolver* prueba el conjunto de reglas en su totalidad. Aun así, se crearon predicados de prueba como *probarRegla0* o *probarRegla1*, para comprobar el correcto funcionamiento de las reglas. Y, además, también se probaron las reglas de manera conjunta, como con *probarReglas0y1*. De esta manera, se fue depurando el código y verificando que el funcionamiento de las reglas fuese el correcto.

Validación del Conjunto de Ejemplos

Para la validación de ejemplos se fueron resolviendo individualmente todos los sudokus para así saber cuáles se lograban resolver con éxito y cuales daban problemas y, por lo tanto, indicaban alguna ligera necesidad modificación en el código. La resolución

individual de los sudokus también ha servido para verificar que los resultados obtenidos más tarde en métricas fueran coherentes y no estuviese mostrando datos falsos o erróneos.

El conjunto de sudokus está formado por 8 sudokus de cada nivel: básico, intermedio y avanzado. Abarcando así distintos niveles de dificultad que se comprobarían posteriormente en el apartado de métricas.

Además de este conjunto de 24 sudokus, se han añadido dos sudokus extra con determinada peculiaridad. Por una parte, encontramos sudoku_VACIO que consta de un sudoku sin ningún número definido inicialmente en el tablero y por tanto, compuesto únicamente de ceros. En la siguiente imagen se observa cómo existen todas las posibilidades para todas las casillas:

```
----------------------------------------
Sudoku inicial con posibilidades en las casillas vacias:

[1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9]
[1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9]
[1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9]
[1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9]
[1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9]
[1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9]
[1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9]
[1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9]
[1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9]
[1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9] [1,2,3,4,5,6,7,8,9]
```

Por otra parte, se define sudoku_MALO que es un sudoku que nunca podrá ser resuelto ya que viola desde un inicio las reglas propias del rompecabezas. Como se detalla posteriormente, se comprueba si el sudoku es imposible y muestra este mensaje por pantalla:

```
|    resolver(sudoku_MALO).
El Sudoku es imposible de resolver o porque hay una casilla vacia (sin posibilidades) o porque hay algun elemento repetido en la misma fila/columna/cuadrante 3x3.
true
```

Otros aspectos relevantes de la implementación

Además de la lógica principal, previamente explicada, se han implementado diversos aspectos adicionales relevantes que enriquecen la solución planteada:

- **Predicados de impresión en ASCII**: Se dispone de predicados auxiliares como imprimirTableroASCII para ofrecer una visualización clara del estado del sudoku en cualquier fase del proceso de simplificación, especialmente útil durante la depuración o presentación de resultados intermedios.

- **Comprobación inmediata de imposibilidad**: Como hemos explicado antes, el sistema cuenta con un mecanismo preventivo de validación (*es_imposible*) que detectan inmediatamente casos como casillas vacías de posibilidades o contradicciones internas, como pueden ser valores repetidos en la misma fila, columna o cuadrante 3x3, evitando así ejecutar pasos adicionales que serían infructuosos.

- **Métricas detalladas**: El módulo metricas.pl ofrece predicados para recopilar información exhaustiva y cuantitativa acerca del comportamiento del sistema, permitiendo no sólo validar resultados sino optimizar futuras mejoras basadas en datos reales observados por nosotros.

- **Flexibilidad en la ejecución de reglas**: Se ha dotado al sistema de una implementación modular, en la que las reglas de simplificación pueden ejecutarse de forma independiente, secuencial, o incluso inversa. Esto permite explorar diferentes enfoques y estrategias en función de los objetivos de simplificación o resolución planteados.

Estos aspectos destacan la modularidad y robustez de la solución propuesta, ofreciendo un sistema completo y extensible para el análisis y simplificación de sudokus en Prolog.

Mejoras y Extensiones

En total, se han realizado tres mejoras o extensiones:

1. Interfaz ASCII

Con esta interfaz el usuario puede usuario visualizar el tablero antes y después de aplicar cada regla. Esta interfaz tiene como objetivo mejorar la comprensión del estado del juego en cada paso, facilitando el seguimiento de los cambios en el tablero y facilita la experiencia del usuario al mostrar claramente los valores resueltos y las posibilidades restantes.

La función imprimirTableroASCII, aplica una estructura de cuadros con bordes que simula la cuadrícula clásica del Sudoku. Añade “-” y “|” hasta conseguir las líneas verticales y horizontales que conforman los cuadros del Sudoku.

```
+-------+-------+-------+
| 4 3 5 | 2 6 9 | 7 8 1 |
| 6 8 2 | 5 7 1 | 4 9 3 |
| 1 9 7 | 8 3 4 | 5 6 2 |
+-------+-------+-------+
| 8 2 6 | 1 9 5 | 3 4 7 |
| 3 7 4 | 6 8 2 | 9 1 5 |
| 9 5 1 | 7 4 3 | 6 2 8 |
+-------+-------+-------+
| 5 1 9 | 3 2 6 | 8 7 4 |
| 2 4 8 | 9 5 7 | 1 3 6 |
| 7 6 3 | 4 1 8 | 2 5 9 |
+-------+-------+-------+
```

Sudoku resuelto en formato ASCII

2. Detección de conflictos

Mediante esta mejora conseguimos implementar un sistema automático que verifique si el Sudoku en su estado actual presenta conflictos o si es irresoluble. El objetivo es detectar de forma temprana situaciones inválidas, como celdas sin posibilidades o números repetidos en filas, columnas o bloques. Esta detección no solo ayuda a depurar el proceso de resolución, sino que también evita que el sistema continúe procesando tableros imposibles, optimizando el rendimiento.

Utilizamos la función es_imposible/1 que comprueba si alguna casilla del tablero de posibilidades se ha quedado sin opciones y si existen repeticiones de valores en filas, columnas y/o cuadros, con hay_repetidos/1.

La función hay_repetidos/1 a su vez utiliza tres predicados auxiliares (filas_invalidas/1, columnas_invalidas/1, cuadros_invalidos/1) que recorren cada región del Sudoku utilizando el predicado rango/3 para iterar sobre índices. En cada recorrido, extraen solo los valores resueltos y únicos usando valores_unicos/2, y luego verifican si hay repeticiones mediante tiene_repetidos/1.

La detección de repetidos se realiza comparando cada valor de la lista contra el resto usando contarApariciones2/3, asegurando que no haya valores duplicados en una misma región.

3. Iteración

Esta mejora consiste en automatizar la aplicación repetitiva de las reglas de simplificación del Sudoku hasta alcanzar un punto de estabilidad, es decir, hasta que aplicar nuevamente las reglas no genere ningún cambio en el tablero de posibilidades. Para implementar esta mejora se usa *simplificar_hasta_estabilizar.* Este predicado aplica secuencialmente las Reglas 0, 1, 2 y 3, imprimiendo el estado del tablero tras cada regla y mostrando el número de iteración. La recursión se mantiene hasta que:

- Se detecta un Sudoku imposible (es_imposible/1).
- No se producen más cambios en el tablero, alcanzando la estabilidad.

Métricas

Para la realización de este apartado se han elaborado varios predicados que, usando las reglas, miden:

- Cálculo de métricas sobre un sudoku individual
- Aplicación de secuencias de reglas de resolución y evaluación de su efectividad.
- Contabilización de la cantidad de celdas resueltas por cada regla.
- Análisis de la eficiencia de dos flujos de resolución: flujo normal y flujo inverso.
- Cálculo de las estadísticas globales sobre una lista de Sudokus.

Las métricas individuales que se miden en cada sudoku son:

- Número de casillas vacías iniciales.
- Cantidad de celdas resueltas por cada regla (R0, R1, R2, R3).
- Porcentaje de efectividad de cada regla sobre el total de vacías.
- Celdas resueltas y porcentaje con flujo normal e inverso.
- Número de iteraciones por cada flujo.
- Estado final (resuelto o no) para cada flujo.

Las métricas globales al procesar una lista de sudokus son:

- Suma total de casillas vacías y celdas resueltas por cada regla.
- Total y porcentaje de celdas resueltas por flujo.
- Total de iteraciones por flujo.
- Promedio de iteraciones por Sudoku.
- Número y porcentaje de Sudokus resueltos por cada flujo.

Tras el planteamiento de las métricas diseñadas para la evaluación de la efectividad y de la eficiencia de las estrategias de simplificación, se han planteado los dos flujos de aplicación previamente mencionados normal (R0 → R1 → R2 → R3) e inverso (R3 → R2 → R1 → R0).

Las métricas se calculan a través de un predicado principal, calcular_metricas/12 que analiza el progreso de la resolución de un Sudoku desde su estado inicial hasta su estabilización o resolución completa.

1. Total de Casillas Vacías Iniciales (TotalVacias)

La métrica TotalVacias mide el número de casillas vacías presentes en el Sudoku inicial, calculado por el predicado auxiliar contar_vacias/2. Este recorre la representación plana del tablero (lista de 81 elementos) y cuenta las apariciones del valor 0, que indica una casilla sin valor asignado.

Esta métrica es esencial como punto de referencia, ya que mide la cantidad de casillas que deben resolverse para completar el Sudoku. En el análisis, se utiliza como denominador para calcular porcentajes de resolución, permitiendo evaluar el impacto relativo de cada regla o flujo sobre el total de incógnitas iniciales.

2. Casillas Resueltas por Reglas Individuales

(ResueltasR0FINAL, ResueltasR1FINAL, ResueltasR2FINAL, ResueltasR3FINAL)

Estas métricas cuantifican el número de casillas vacías resueltas al aplicar exclusivamente cada una de las reglas (R0, R1, R2 y R3) en una única pasada sobre el tablero de posibilidades inicial (TP).

El cálculo se realiza mediante metricas_regla/3, que invoca la regla correspondiente (regla0/4, regla1/3, regla2/3 o regla3/3) y utiliza contar_resueltas/3 para determinar cuántas casillas pasan de tener varias posibilidades (o ninguna) a una única posibilidad.

Para obtener el valor final (FINAL), se ajusta el resultado restando las casillas inicialmente llenas (81 - TotalVacias), enfocándose únicamente en las vacías resueltas.

De tal manera, conseguimos evaluar la potencia de cada regla de forma aislada, destacando su contribución específica al proceso de resolución.

3. Casillas Resueltas por Flujos Combinados

(ResueltasFlujoNormal y ResueltasFlujoInverso)

Las métricas ResueltasFlujoNormal y ResueltasFlujoInverso miden el número de casillas vacías resueltas al aplicar iterativamente las cuatro reglas en dos órdenes distintos hasta que el tablero se estabiliza o se detecta una imposibilidad.

El flujo normal sigue la secuencia R0 → R1 → R2 → R3, implementada en simplificar_hasta_estabilizar_metricas/4, mientras que el flujo inverso usa R3 → R2 → R1 → R0, implementada en simplificar_hasta_estabilizar_inverso/4.

El contador se realiza con contar_resueltas_totales/3, ajustado nuevamente por 81 - TotalVacias.

Con esta métrica podemos medir la eficacia de combinar las reglas en un enfoque iterativo, permitiendo comparar cómo el orden de aplicación afecta el progreso en la resolución. Port ejemplo, un valor elevado en ResueltasFlujoNormal frente a ResueltasFlujoInverso podría sugerir que las reglas más simples (como R0) son más efectivas como paso inicial en el Sudoku dado.

4. Número de Iteraciones

(IteracionesNormal e IteracionesInverso)

Estas métricas registran el número de ciclos completos de aplicación de las reglas necesarios para estabilizar el tablero en cada flujo.

En los predicados simplificar_hasta_estabilizar_metricas/4 y simplificar_hasta_estabilizar_inverso/4, una iteración se cuenta cada vez que se aplica el flujo completo y el tablero resultante difiere del anterior (NTP3 \== TP).

El proceso termina cuando el tablero no cambia más, se detecta una imposibilidad (es_imposible/1), o se resuelve completamente.

Con esto, conseguimos un indicador de la complejidad computacional de cada estrategia, ya que un mayor número de iteraciones implica un esfuerzo mayor para alcanzar la estabilización. Es decir, si el orden de las reglas influye en la eficiencia de la resolución-

5. Estado de Resolución

(ResueltoNormal y ResueltoInverso)

Las métricas ResueltoNormal y ResueltoInverso son valores binarios (1 para "sí", 0 para "no") que indican si el Sudoku se resolvió completamente con el flujo normal o inverso, respectivamente.

El predicado esta_resuelto/1 verifica esta condición comprobando que todas las casillas del tablero final tengan exactamente una posibilidad (longitud 1). Un valor de 1 significa que el flujo logró asignar un valor único a cada casilla, resolviendo el Sudoku, mientras que un valor de 0 indica que el proceso falló, ya sea por una contradicción o por casillas sin resolver.

Estas métricas evaluan el éxito global de cada enfoque, pudiendo determinar si un Sudoku puede resolverse con las reglas definidas.

Sea, por ejemplo, el sudoku_AVANZADO_3 ANALIZADO:

```
17 ?- probar_metricas(sudoku_AVANZADO_3).
MÃ©tricas para sudoku_AVANZADO_3:
Total de casillas vacÃ as iniciales: 58
Casillas resueltas solo con Regla 0: 0 (0%)
Casillas resueltas solo con Regla 1: 1 (1.7241379310344827%)
Casillas resueltas solo con Regla 2: 0 (0%)
Casillas resueltas solo con Regla 3: 0 (0%)
Casillas resueltas con el flujo normal (R0+R1+R2+R3): 2 (3.4482758620689653%)
Casillas resueltas con el flujo inverso (R3+R2+R1+R0): 2 (3.4482758620689653%)
NÃºmero de iteraciones con el flujo normal (R0+R1+R2+R3): 4
NÃºmero de iteraciones con el flujo inverso (R3+R2+R1+R0): 3
Resuelto con el flujo normal (R0+R1+R2+R3): No
Resuelto con el flujo inverso (R3+R2+R1+R0): No
true
```

El Sudoku inicia con 58 casillas vacías, lo que representa más del 70% del tablero (81 - 58 = 23 casillas llenas), indicando un nivel de dificultad elevado. Las reglas individuales muestran un desempeño casi nulo: la Regla 0, 2 y 3 no resuelven ninguna casilla (0%), mientras que la Regla 1 resuelve solo 1 (1.724%), destacando la resistencia del tablero a técnicas básicas aplicadas aisladamente. Los flujos combinados, normal (R0+R1+R2+R3) e inverso (R3+R2+R1+R0), resuelven apenas 2 casillas cada uno (3.448%), duplicando el progreso individual pero aún insuficiente para avanzar significativamente. Además, el flujo normal requiere 4 iteraciones y el inverso 3, sugiriendo una estabilización rápida pero con resultados limitados, ya que ambos flujos terminan sin resolver el Sudoku completo ("No" en ambos casos).

La falta de resolución y el bajo número de casillas resueltas (2 de 58) indican que las reglas implementadas (R0 a R3) no abordan patrones complejos necesarios para este tablero, a pesar de las iteraciones múltiples. La similitud entre los flujos normal e inverso en casillas resueltas sugiere que el orden de aplicación tiene un impacto mínimo, aunque el inverso es ligeramente más eficiente (3 vs 4 iteraciones).

Probamos para un sudoku más simple, como sudoku_BASICO_2:

```
2 ?- probar_metricas(sudoku_BASICO_2).
MÃ©tricas para sudoku_BASICO_2:
Total de casillas vacÃ as iniciales: 1
Casillas resueltas solo con Regla 0: 1 (100%)
Casillas resueltas solo con Regla 1: 1 (100%)
Casillas resueltas solo con Regla 2: 1 (100%)
Casillas resueltas solo con Regla 3: 1 (100%)
Casillas resueltas con el flujo normal (R0+R1+R2+R3): 1 (100%)
Casillas resueltas con el flujo inverso (R3+R2+R1+R0): 1 (100%)
NÃºmero de iteraciones con el flujo normal (R0+R1+R2+R3): 1
NÃºmero de iteraciones con el flujo inverso (R3+R2+R1+R0): 1
Resuelto con el flujo normal (R0+R1+R2+R3): SÃ
Resuelto con el flujo inverso (R3+R2+R1+R0): SÃ
```

Comparado con sudoku_AVANZADO_3, que inicia con 58 casillas vacías, sudoku_BASICO_2 presenta una complejidad menor. Mientras que las reglas individuales en el avanzado resuelven casi nada (0-1 casillas, 0-1.724%), en el básico todas resuelven el 100% de la única casilla vacía.

Los flujos en sudoku_AVANZADO_3 resuelven solo 2 casillas (3.448%) en 4 y 3 iteraciones, sin completar el Sudoku, frente al 100% en una iteración para sudoku_BASICO_2.

Esto confirma que el solucionador es muy efectivo para los Sudokus básicos, pero es muy limitado con los avanzados, sugiriendo que la cantidad de casillas vacías y la complejidad de los patrones son factores clave en su rendimiento, y que técnicas adicionales serían necesarias para casos como sudoku_AVANZADO_3.

Comprobamos ahora las métricas globales con listas de sudokus:

```
[1] 3 ?- probar_metricas_globales.
=== MÃ©tricas globales para todos los sudokus ===
NÃºmero de sudokus analizados: 24
Total de casillas vacÃ as iniciales: 825
Casillas resueltas solo con Regla 0: 241 (29.21212121212121%)
Casillas resueltas solo con Regla 1: 144 (17.454545454545457%)
Casillas resueltas solo con Regla 2: 45 (5.454545454545454%)
Casillas resueltas solo con Regla 3: 65 (7.878787878787878%)
Casillas resueltas con el flujo normal (R0+R1+R2+R3): 346 (41.93939393939394%)
Casillas resueltas con el flujo inverso (R3+R2+R1+R0): 346 (41.93939393939394%)
NÃºmero total de iteraciones con el flujo normal (R0+R1+R2+R3): 50
Media de iteraciones por sudoku con el flujo normal: 2.0833333333333335
NÃºmero total de iteraciones con el flujo inverso (R3+R2+R1+R0): 41
Media de iteraciones por sudoku con el flujo inverso: 1.7083333333333333
NÃºmero total de sudokus resueltos con el flujo normal (R0+R1+R2+R3): 16 (66.66666666666666%)
NÃºmero total de sudokus resueltos con el flujo inverso (R3+R2+R1+R0): 16 (66.66666666666666%)
true
```

Se miden en total 24 sudokus. En promedio, cada Sudoku tiene aproximadamente 34.375 casillas vacías (825 / 24).

Las reglas individuales muestran desempeños variados: la Regla 0 resuelve 241 casillas (29.21%), destacándose como la más efectiva en solitario, seguida por la Regla 1 con 144 (17.45%), la Regla 3 con 65 (7.88%) y la Regla 2 con 45 (5.45%). Esto indica que las técnicas más básicas (R0 y R1) tienen un impacto significativo en la reducción inicial de

posibilidades, mientras que las más avanzadas (R2 y R3) son menos aplicables aisladamente.

Los flujos combinados, normal (R0+R1+R2+R3) e inverso (R3+R2+R1+R0), resuelven 346 casillas cada uno (41.94%), superando la suma de las reglas individuales (495 casillas brutas). Concluimos que las interacciones iterativas entre reglas potencian el progreso, aunque solo resuelven el 42% del total de vacías.

El orden inverso usa 41 iteraciones frente al normal, que requiere 50. Lo que indica que las reglas avanzadas preparan el tablero para las simples. Ambos flujos resuelven 16 tableros y dejan 8 sin resolver, por lo que el orden no altera el resultado final, solo se produce una mejora computacional.

Aplicamos ahora las métricas para los conjuntos de sudokus individuales, es decir, básicos, intermedio y avanzado:

```
[2] 4 ?-
probar_metricas_globales_basicos.
=== MÃ©tricas globales para todos los sudokus ===
NÃºmero de sudokus analizados: 8
Total de casillas vacÃ as iniciales: 12
Casillas resueltas solo con Regla 0: 12 (100%)
Casillas resueltas solo con Regla 1: 12 (100%)
Casillas resueltas solo con Regla 2: 12 (100%)
Casillas resueltas solo con Regla 3: 12 (100%)
Casillas resueltas con el flujo normal (R0+R1+R2+R3): 12 (100%)
Casillas resueltas con el flujo inverso (R3+R2+R1+R0): 12 (100%)
NÃºmero total de iteraciones con el flujo normal (R0+R1+R2+R3): 8
Media de iteraciones por sudoku con el flujo normal: 1
NÃºmero total de iteraciones con el flujo inverso (R3+R2+R1+R0): 8
Media de iteraciones por sudoku con el flujo inverso: 1
NÃºmero total de sudokus resueltos con el flujo normal (R0+R1+R2+R3): 8 (100%)
NÃºmero total de sudokus resueltos con el flujo inverso (R3+R2+R1+R0): 8 (100%)
true
% Break level 3
[3] 5 ?-
probar_metricas_globales_intermedios.
=== MÃ©tricas globales para todos los sudokus ===
NÃºmero de sudokus analizados: 6
Total de casillas vacÃ as iniciales: 223
Casillas resueltas solo con Regla 0: 127 (56.95067264573991%)
Casillas resueltas solo con Regla 1: 81 (36.32286995515695%)
Casillas resueltas solo con Regla 2: 25 (11.210762331838566%)
Casillas resueltas solo con Regla 3: 37 (16.59192825112107 5%)
Casillas resueltas con el flujo normal (R0+R1+R2+R3): 223 (100%)
Casillas resueltas con el flujo inverso (R3+R2+R1+R0): 223 (100%)
NÃºmero total de iteraciones con el flujo normal (R0+R1+R2+R3): 18
Media de iteraciones por sudoku con el flujo normal: 3
NÃºmero total de iteraciones con el flujo inverso (R3+R2+R1+R0): 14
Media de iteraciones por sudoku con el flujo inverso: 2.3333333333333335
NÃºmero total de sudokus resueltos con el flujo normal (R0+R1+R2+R3): 6 (100%)
NÃºmero total de sudokus resueltos con el flujo inverso (R3+R2+R1+R0): 6 (100%)
true
% Break level 4
[4] 6 ?- probar_metricas_globales_avanzados.
=== MÃ©tricas globales para todos los sudokus ===
NÃºmero de sudokus analizados: 8
Total de casillas vacÃ as iniciales: 466
Casillas resueltas solo con Regla 0: 102 (21.888412017167383%)
Casillas resueltas solo con Regla 1: 49 (10.515021459227468%)
Casillas resueltas solo con Regla 2: 8 (1.7167381974248928%)
Casillas resueltas solo con Regla 3: 16 (3.4334763948497855%)
Casillas resueltas con el flujo normal (R0+R1+R2+R3): 108 (23.17596566523605%)
Casillas resueltas con el flujo inverso (R3+R2+R1+R0): 108 (23.17596566523605%)
NÃºmero total de iteraciones con el flujo normal (R0+R1+R2+R3): 17
Media de iteraciones por sudoku con el flujo normal: 2.125
NÃºmero total de iteraciones con el flujo inverso (R3+R2+R1+R0): 14
Media de iteraciones por sudoku con el flujo inverso: 1.75
NÃºmero total de sudokus resueltos con el flujo normal (R0+R1+R2+R3): 2 (25.0%)
NÃºmero total de sudokus resueltos con el flujo inverso (R3+R2+R1+R0): 2 (25.0%)
true
```

Para los 8 Sudokus básicos, con solo 12 casillas vacías (1.5 por Sudoku), todas las reglas individuales (R0 a R3) y ambos flujos, normal e inverso, resuelven el 100% de las casillas en una iteración por Sudoku (media de 1), logrando resolver todos los tableros (8, 100%).

Los 6 Sudokus intermedios, con 223 casillas vacías (37.17 por Sudoku), muestran una mayor dificultad: las reglas individuales resuelven entre el 11.21% (R2) y el 56.95% (R0), pero los flujos combinados alcanzan el 100% (223 casillas), requiriendo 3 iteraciones promedio en el flujo normal y 2.33 en el inverso, resolviendo todos los tableros (6, 100%). Mostrando que las reglas son suficientes para intermedios mediante iteraciones.

Por último, los 8 Sudokus avanzados, con 466 casillas vacías (58.25 por Sudoku), muestran limitaciones: las reglas individuales resuelven del 1.72% (R2) al 21.89% (R0), y los flujos solo alcanzan el 23.18% (108 casillas), con 2.125 y 1.75 iteraciones promedio, resolviendo apenas 2 Sudokus (25%), como se vio en el análisis individual del sudoku_AVANZADO_3.

De aquí concluimos que la herramienta de resolución de sudokus es altamente eficiente para los sudokus básicos e intermedios, pero muestra muchos fallos en los avanzados. La regla 0 es la más efectiva en todos los niveles y que las reglas 2 y 3 tienen menos relevancia. Además, los flujos de iteración de las reglas incrementan las posibilidades de resolución de los sudokus. El flujo inverso es más eficiente pues muestra menos iteraciones, pero no mejora los resultados.

Por último, observamos que las reglas actuales son insuficientes al incrementar la complejidad del sudoku, indicando la necesidad de implementación de nuevas técnicas como backtracking y así poder abordar el 77% de las casillas vacías no resueltas.

Fuentes consultadas

Para la realización de esta práctica, se han seguido las pautas indicadas el en enunciado proporcionado, en el que se exponen con precisión y claridad los puntos a seguir y los requerimientos exigidos.

También nos ha sido de gran utilidad la rúbrica de evaluación de la práctica para hacernos una mejor idea de los puntos imprescindibles a implementar y la calificación y valoración de todos los aspectos referentes al trabajo.

En relación con aspectos de implementación de código, hemos hecho uso del documento de introducción a la programación en Prolog, así como de los tres archivos sobre el manejo de listas con y sin funciones nativas y el ejemplo de código sobre libros en Prolog, todos ellos subidos en el aula virtual de la asignatura.

A demás de lo adjunto en la plataforma, nos hemos servido del manual online de Prolog. Se puede acceder al mismo a través del siguiente enlace: https://swish.swi-prolog.org/p/Tutorial%20de%20prolog.swinb

En cuanto a la obtención de sudokus válidos, de diferentes niveles de complejidad, que hemos ido obteniendo para conformar nuestro banco de pruebas, hemos hecho uso de los siguientes enlaces o fuentes, en las que se puede trabajar con sudokus, obtenerlos y comprobar resultados:

- https://www.livesudoku.com/es/sudoku/medium/
- https://www.sudoku-online.org/imprimir-sudokus.php
- https://sudoku.com/es/sudoku-solver

Distribución de tareas

La distribución de las tareas a realizar por los miembros del grupo se ha ido realizando conforme se iba avanzando en la resolución y ejecución de la práctica. En lugar de asignar desde el principio tareas fijas e inamovibles a cada integrante, hemos optado por una metodología de trabajo colaborativa y flexible, en la que cada miembro ha ido asumiendo diferentes funciones según las necesidades del momento. Esta dinámica ha permitido un desarrollo más eficiente del trabajo, ya que cada integrante ha podido aportar sus conocimientos y habilidades en distintas áreas conforme surgían los retos.

La asignación personal y precisa de cada procedimiento, entre los miembros del grupo, se ha ido estableciendo y comunicando al profesor a través de las minutas. De esta forma, hemos podido dejar constancia del trabajo progresivo de cada uno de los miembros en un formato escrito enviado al profesor, asegurando la trazabilidad de la evolución de la práctica. Sin embargo, más allá de esta asignación formal, el trabajo ha sido en todo momento un esfuerzo conjunto, caracterizado por la cooperación y el apoyo mutuo.

A lo largo de la práctica, hemos trabajado de manera colectiva en todos los aspectos del proyecto, ayudándonos unos a otros en la resolución de problemas, la toma de decisiones y la implementación de soluciones. Esta forma de trabajo ha permitido que todos los miembros del grupo se involucren en cada una de las fases del desarrollo, enriqueciendo el proceso con diferentes perspectivas y asegurando un mejor entendimiento global del proyecto. Debido a esta constante colaboración, la individualización de tareas resulta difícil, ya que cada miembro ha participado activamente en distintas áreas en función de las necesidades que iban surgiendo. La interacción continua y la retroalimentación entre nosotros han sido clave para el éxito del trabajo, fomentando un aprendizaje más integral y una mayor cohesión dentro del equipo.

Conclusiones

En el desarrollo de esta práctica se ha logrado alcanzar con éxito los objetivos establecidos inicialmente. Se ha trabajado intensivamente en la manipulación eficiente de listas en Prolog, profundizando en conceptos clave para la representación y simplificación progresiva de Sudokus, empleando estructuras lineales de manera efectiva.

La implementación del sudoku como una lista lineal de 81 elementos nos ha facilitado la gestión y el procesamiento de datos. El desarrollo de las reglas de simplificación (Regla 0, Regla 1, Regla 2 y Regla 3) ha permitido comprender claramente la importancia de la interacción entre estas reglas y cómo su aplicación puede resolver o simplificar considerablemente distintos niveles de dificultad en sudokus.

A partir de la validación del conjunto de ejemplos, se ha comprobado que la combinación secuencial de reglas (flujo normal) ofrece resultados más efectivos en términos generales, logrando resolver completamente un número significativo de casos propuestos. Sin embargo, también se observó que la aplicación inversa de las reglas (flujo inverso) resulta interesante como método alternativo, aunque generalmente menos efectivo en cuanto al número de casillas resueltas. No siempre es así ya que en algunos sudokus se obtienen resultados muy similares en cuanto a número de casillas resueltas por ambos métodos.

Gracias a las métricas desarrolladas en la implementación, ha sido posible obtener una visión clara del rendimiento y efectividad de cada regla por separado, así como del conjunto de reglas en su interacción. Esto ha permitido identificar con precisión en qué medida las reglas básicas pueden resolver los sudokus propuestos, evidenciando que algunos casos de dificultad intermedia y avanzada requieren de reglas adicionales o métodos complementarios para ser completamente resueltos.

Entre los aspectos relevantes del proyecto, cabe destacar la implementación de mejoras significativas, como la interfaz ASCII, que ha facilitado la visualización y seguimiento del proceso de resolución paso a paso.

Finalmente, este proyecto nos ha permitido desarrollar competencias prácticas esenciales en Prolog, como son la gestión de listas, el uso de recursividad, y el diseño e implementación de algoritmos basados en reglas para la simplificación de problemas complejos.

Anexos

Anexo 1: Lista de Variables

- **I:** Índice de la casilla en el tablero (del 0 al 80).
- **F**: Número de fila correspondiente a I.
- **C**: Número de columna correspondiente a I.
- **S:** Índice del cuadro 3x3 correspondiente a I.
- **LF:** Lista con los índices de la fila F.
- **LC:** Lista con los índices de la columna C.
- **LS:** Lista con los índices del cuadro S.
- **LDC**: Lista con los índices de la columna C que no están en la fila F.
- **LFC**: Lista con los índices de la fila F y los de la columna C sin repeticiones.
- **LDS**: Lista con los índices del cuadro S sin los índices de LFC.
- **LL**: Lista combinada de LFC y LDS.
- **TP**: Tablero de Sudoku, representado como una lista de listas (cada lista es una fila).
- **X**: Elemento en la casilla I del tablero (puede ser un número o una lista de posibilidades).
- **NTP**: Nuevo tablero resultante después de realizar una operación (como quitar elementos o reemplazar valores).
- **V**: Lista de valores ya procesados durante la recursión (usada en reglas como la regla0).
- **N**: Número de veces que un elemento aparece en una lista (como en la función contarApariciones).
- **L**: Lista de posibles valores para una casilla vacía.
- **P**: Lista de posibilidades restantes para una casilla después de eliminar los valores presentes en su fila, columna y cuadro.
- **NNTP**: Nuevo tablero parcial resultante después de aplicar una subregla o una operación intermedia en una casilla específica.
- **TNN**: Tablero parcial obtenido en una llamada recursiva durante la generación de posibilidades.
- **TD**: Tablero definitivo con las posibilidades calculadas para todas las casillas en hacerPosibilidades.
- **NL**: Nueva lista de posibilidades tras eliminar un elemento de las posiciones conflictivas.

- **LE**: Lista final de elementos obtenidos al recorrer una lista de índices.
- **IF**: Índice de fila calculado a partir de un índice global I.
- **IC**: Índice de columna calculado a partir de un índice global I.
- **IS**: Índice del cuadro 3x3 calculado a partir de la fila y columna.
- **NI**: Nuevo índice calculado como el siguiente paso en una iteración recursiva.
- **P1**: Lista intermedia de posibilidades tras eliminar valores de la fila.
- **P2**: Lista intermedia de posibilidades tras eliminar valores de la columna.
- **E**: Elemento específico extraído de una posición del tablero.
- **LL**: Longitud de una lista de posibilidades en una casilla.
- **D**: Diferencia entre una lista de posibilidades y otra lista.
- **TA**: Total de apariciones de una lista de posibilidades en una fila, columna o cuadro según la regla 3.
- **NTP1**: Tablero intermedio resultante después de aplicar una regla o subregla en una iteración.
- **NTP2**: Tablero intermedio tras aplicar una segunda regla en una secuencia de simplificación.
- **NTP3**: Tablero intermedio tras aplicar una tercera regla en una secuencia de simplificación.
- **TPFinal**: Tablero final después de aplicar todas las reglas hasta estabilizar ¡.
- **LX**: Longitud de la lista de posibilidades de una casilla en la regla 3.
- **NX**: Nueva lista de posibilidades formada al combinar una lista existente con un número adicional.
- **RN**: Resto de la lista de números posibles para combinar en la regla 3.
- **Valores**: Lista de valores únicos extraídos de una fila, columna o cuadro para verificar repeticiones.
- **X1**: Valor único extraído de una lista de longitud 1 en una casilla.
- **R**: Tablero resultante final después de aplicar una regla recursivamente (regla0, regla1, etc.).
- **LN**: Longitud de la lista de posibilidades en una casilla para la regla 2.
- **T**: Total de apariciones de una pareja o lista de posibilidades en una fila, columna o cuadro.
- **IFS**: Lista de índices de una fila específica.
- **ICS**: Lista de índices de una columna específica.
- **ISS**: Lista de índices de un cuadro 3x3 específico.
- **ISI**: Índice inicial del cuadro 3x3 ajustado para la regla 3.
- **D**: Lista de números posibles excluyendo las posibilidades actuales de una casilla.
- **NX**: Nueva lista de posibilidades extendida con un número adicional para comprobar combinaciones.
- **NT**: Número de apariciones acumuladas en el resto de una lista durante la recursión.

Anexo 2: Lista de funciones

- **sudoku(TP):** Representa un tablero de Sudoku como una lista de 81 elementos, donde cada número indica el valor en la casilla correspondiente (0 significa casilla vacía).
- **mostrarPosibilidades(TP)** Genera y muestra las posibles soluciones para cada casilla vacía en el tablero TP. Llama a hacerPosibilidades/2 para calcularlas y imprimirPosibilidades/1 para mostrarlas.
- **imprimirTablero(TP)** Imprime el tablero de Sudoku en un formato legible. Llama a imprimirFilas/2 para recorrer el tablero fila por fila.
- **imprimirFilas(TP, I)** Imprime fila por fila el tablero TP a partir del índice I. Sustituye los ceros por puntos y organiza la salida en líneas de 9 elementos.
- **imprimirPosibilidades(TP)** Imprime las posibles soluciones de cada casilla vacía del tablero TP, llamando a imprimirPosibilidadesAux/2 para recorrer todas las casillas.
- **imprimirPosibilidadesAux(TP, I)** Recorre e imprime las posibilidades de cada casilla del tablero TP, indicando el índice de la casilla junto con los valores posibles.
- **conflictivos(I, L)** Calcula las posiciones conflictivas de una casilla I en un tablero de Sudoku. Calcula el número de fila (F) y columna (C) de I. Encuentra el cuadro 3x3 donde está I. Encuentra las posiciones en la fila, columna y cuadro, y las combina para obtener todas las casillas conflictivas, excluyendo la propia casilla I.
- **quitarElementoDeConflictivos(TP, [], _, TP)** Recursivamente elimina un elemento E de todas las posiciones conflictivas de la lista de índices, en la estructura del tablero TP. Cuando ya no quedan elementos en la lista de índices, termina la recursión.
- **cogerElementosIndice(TP, [X|Y], L, LE)** Obtiene los valores del tablero TP para una lista de índices [X|Y]. Acumula los valores encontrados en la lista L y devuelve la lista completa LE.
- **fila(T, I, F)** Devuelve todos los elementos de la fila I del tablero T utilizando indicesFila.
- **columna(T, I, C)** Devuelve todos los elementos de la columna I del tablero T utilizando indicesColumna.
- **cuadro(T, I, S)** Devuelve los elementos del cuadro 3x3 correspondiente al índice I, usando indicesCuadro.
- **indicesFila(F, LF)** Genera una lista con los índices de la fila F en el tablero, del 0 al 8.

- **indicesColumna(C, LC)** Genera una lista con los índices de la columna C en el tablero, del 0 al 8.
- **indicesCuadro(S, LS)** Genera una lista con los índices del cuadro 3x3 al que pertenece el índice S.
- **hacerPosibilidades(T, TP)** Inicializa la búsqueda de las posibilidades para cada casilla vacía (que no tiene un número fijo). La función recursiva calcula todas las posibles opciones para cada casilla en el tablero.
- **regla0(TP, I, V, R)** Implementa la Regla 0 de resolución, que implica simplificar el tablero utilizando una lógica de casillas únicas en filas, columnas y cuadros. Si una casilla tiene solo un valor posible, lo fija y actualiza el tablero eliminando ese valor de las casillas conflictivas.
- **regla1(TP, I, NTP)** Implementa la Regla 1 de simplificación, que resuelve las casillas que tienen un solo valor posible en su fila, columna o cuadro.
- **subregla1(TP, I, X, NTP)** Aplica las subreglas de la Regla 1 a una casilla dada, determinando si un número en la casilla es único en su fila, columna o cuadro.
- **regla2(TP, I, NTP)** Implementa la Regla 2, que resuelve las casillas en las que solo hay dos posibilidades, y esas dos posibilidades aparecen exactamente en dos casillas de la misma fila, columna o cuadro. Elimina esas posibilidades de las demás casillas.
- **subregla2(TP, I, L, NTP)** Aplica la subregla de la Regla 2 en una casilla, cuando hay dos posibles valores, y esos valores están presentes en exactamente dos casillas de la misma fila, columna o cuadro.
- **regla3(TP, I, NTP)** Simplifica un tablero de posibilidades TP utilizando la regla 3, revisando casillas con 2 o 3 posibilidades y buscando trios en filas, columnas o cuadros que permitan eliminar otras posibilidades. Llama a subregla3/4 y recorre el tablero desde el índice I hasta 81.
- **subregla3**(TP, I, X, NTP) simplifica un tablero de Sudoku (TP) aplicando la Regla 3 en la casilla I con posibilidades X, buscando "Naked Triples". Analiza casillas con 2 o 3 posibilidades, identificando tríos en filas, columnas o cuadros para eliminar esos valores de otras casillas
- **comprobarCombinacionesRegla3(TP, I, X, RN, NTP)** Para una casilla I con posibilidades X, prueba combinaciones añadiendo números de la lista RN uno por uno, aplicando subregla3 a cada combinación para verificar si forma un trio válido, y devuelve el tablero actualizado NTP.
- **quitarListaRegla3(TP, Indices, L, NTP)** Elimina los elementos de la lista L de las casillas indicadas por Índices en el tablero TP, pero solo si la diferencia entre las posibilidades actuales y L no es vacía, devolviendo el nuevo tablero NTP.
- **contarAparicionesRegla3(L, E, T)** Cuenta cuántas veces una lista de posibilidades E está completamente contenida

en las sublistas de L (es decir, la diferencia entre una sublista y E es vacía), devolviendo el total en T. Usado en la regla 3 para identificar triplets.

- **imprimirTableroASCII(TP)** Imprime el tablero de Sudoku en un formato ASCII con líneas separadoras para visualizar mejor las filas, columnas y cuadros 3x3. Llama a imprimirFilasASCII/2 para recorrer el tablero.
- **imprimirFilasASCII(TP, I)** Predicado auxiliar que imprime fila por fila el tablero TP en formato ASCII, empezando desde el índice I. Usa separadores verticales y horizontales cada tres casillas y sustituye ceros por puntos.
- **probarImprimirTableroASCII(T)** Función de prueba que llama a imprimirTableroASCII/1 para mostrar el tablero T en formato ASCII.
- **probarRegla0(T)** Aplica la regla 0 al tablero T tras calcular sus posibilidades y muestra el resultado usando imprimirTablero.
- **probarRegla1(T)** Aplica la regla 1 al tablero T tras calcular sus posibilidades y muestra el resultado usando imprimirTablero.
- **probarRegla0y1(T)** Aplica secuencialmente las reglas 0 y 1 al tablero T tras calcular sus posibilidades y muestra el resultado final.
- **probarRegla2(T)** Aplica la regla 2 al tablero T tras calcular sus posibilidades y muestra el resultado usando imprimirTablero.
- **probarRegla3(T)** Aplica la regla 3 al tablero T tras calcular sus posibilidades y muestra el resultado usando imprimirTablero.
- **es_imposible(TP)** Verifica si el tablero TP es imposible de resolver, ya sea porque contiene una casilla sin posibilidades (lista vacía) o porque hay valores repetidos en filas, columnas o cuadros.
- **hay_repetidos(TP)** Comprueba si hay valores repetidos en las filas, columnas o cuadros del tablero TP, llamando a filas_invalidas, columnas_invalidas y cuadros_invalidos.
- **filas_invalidas(TP)** Verifica si alguna fila del tablero TP tiene valores repetidos, iterando sobre las filas y usando valores_unicos y tiene_repetidos.
- **columnas_invalidas(TP)** Verifica si alguna columna del tablero TP tiene valores repetidos, iterando sobre las columnas y usando valores_unicos y tiene_repetidos.
- **cuadros_invalidos(TP)** Verifica si algún cuadro 3x3 del tablero TP tiene valores repetidos, iterando sobre los cuadros y usando valores_unicos y tiene_repetidos.
- **rango(Inicio, Fin, X) Genera** un rango de valores desde Inicio hasta Fin, devolviendo cada valor X en el rango para iteraciones.
- **tiene_repetidos(L)** Comprueba si una lista L contiene valores repetidos, contando apariciones con contarApariciones2.
- **contarApariciones2(L, E, N)** Cuenta cuántas veces aparece un elemento E en una lista L, devolviendo el total en N (usado para verificar repeticiones).

- **valores_unicos(L, Valores) Extrae** los valores únicos de una lista L (ignorando listas de posibilidades y ceros), devolviéndolos en Valores.
- **simplificar_hasta_estabilizar(TP, TPFinal, Paso)** Aplica iterativamente las reglas 0, 1, 2 y 3 al tablero TP hasta que no haya más cambios o sea imposible, mostrando cada paso y devolviendo el tablero final TPFinal.
- **resolver (NombreSudoku)** Resuelve un Sudoku dado su nombre, calculando posibilidades iniciales, aplicando reglas hasta estabilizar, y mostrando el proceso y el resultado final en formato ASCII.
- **contarSemejantes(L, E, T)** Cuenta cuántas veces aparece exactamente un elemento E (como una lista) en una lista L, devolviendo el total en T (usado en la regla 2).
- **quitarLista(TP, Indices, L, NTP)** Elimina una lista L de posibilidades de las casillas indicadas por Indices en el tablero TP, devolviendo el nuevo tablero NTP (usado en la regla 2).
- **comprobarCombinacionesRegla3(TP, I, X, RN, NTP)** Comprueba combinaciones de números adicionales con las posibilidades X de una casilla I, aplicando la regla 3 y devolviendo el tablero actualizado NTP.
- **subregla3(TP, I, X, NTP)** Aplica la regla 3 a una casilla I con posibilidades X, buscando triplets en fila, columna o cuadro que permitan simplificación.
- **quitarListaRegla3(TP, Indices, L, NTP)** Elimina los elementos de la lista L de las casillas indicadas por Indices en el tablero TP, solo si la diferencia no es vacía, devolviendo NTP (usado en la regla 3).
- **contarAparicionesRegla3(L, E, T)** Cuenta cuántas veces una lista E está contenida en las sublistas de L (diferencia vacía), devolviendo el total en T (usado en la regla 3).
- **calcular_metricas(Sudoku, TotalVacias, ResueltasR0FINAL, ResueltasR1FINAL, ResueltasR2FINAL, ResueltasR3FINAL, ResueltasFlujoNormal, ResueltasFlujoInverso, IteracionesNormal, IteracionesInverso, ResueltoNormal, ResueltoInverso)** Calcula métricas de resolución para un Sudoku, incluyendo casillas vacías, resueltas por cada regla, flujos normal e inverso, iteraciones y estado de resolución.
- **probar_metricas(Nombre)** Muestra las métricas detalladas de un Sudoku específico, incluyendo porcentajes de resolución por regla y flujos.
- **calcular_metricas_globales(ListaSudokus)** Calcula y muestra métricas globales para una lista de Sudokus, como totales y promedios de casillas resueltas, iteraciones y resolución.
- **simplificar_hasta_estabilizar_metricas(TP, TPFinal, Paso, Iteraciones)** Simula el flujo normal (R0+R1+R2+R3) sin imprimir, contando iteraciones hasta estabilizar o detectar imposibilidad.

- **simplificar_hasta_estabilizar_inverso(TP, TPFinal, Paso, Iteraciones)** Simula el flujo inverso (R3+R2+R1+R0) sin imprimir, contando iteraciones hasta estabilizar o detectar imposibilidad.
- **esta_resuelto(TP)** Verifica si un tablero TP está completamente resuelto (cada casilla tiene exactamente un valor).
- **contar_vacias(Sudoku, Total)** Cuenta el número total de casillas vacías (ceros) en un tablero Sudoku.
- **contar_resueltas_iniciales(TP, Total)** Cuenta el número de casillas con un solo valor en el tablero de posibilidades inicial TP.
- **metricas_regla(TP, Regla, Resueltas)** Calcula cuántas casillas se resuelven aplicando una regla específica (0, 1, 2 o 3) al tablero TP.
- **contar_resueltas_totales(TPInicial, TPFinal, Total)** Cuenta el número total de casillas resueltas comparando un tablero inicial y final.
- **probar_metricas_globales** Ejecuta calcular_metricas_globales para una lista predefinida de Sudokus de todos los niveles.
- **probar_metricas_globales_basicos** Ejecuta calcular_metricas_globales para Sudokus de nivel básico.
- **probar_metricas_globales_intermedios** Ejecuta calcular_metricas_globales para Sudokus de nivel intermedio.
- **probar_metricas_globales_avanzados** Ejecuta calcular_metricas_globales para Sudokus de nivel avanzado.
- **contar_iteraciones(Sudoku, Iteraciones)** Cuenta el número de iteraciones necesarias para resolver un Sudoku con el flujo normal.